

3_serwisy_ćwiczenia

March 28, 2022

1 Terminal

1.0.1 Zadanie 1

Wyświetlić informacje o serwisach dla symulacji turtlesim. Serwisy:

- /clear - wyczyszczenie narysowanych ścieżek
- /kill - usunięcie robota
- /reset - reset środowiska do stanu początkowego
- /spawn - utworzenie nowego robota
- /turtle1/set_pen - ustawienie koloru pędzla do rysowania
- /turtle1/teleport_absolute - natychmiastowe przeniesienie robota do wskazanej lokalizacji
- /turtle1/teleport_relative - przeniesienie robota, współrzędne podawane w układzie robota

[]:

1.0.2 Zadanie 2

Utworzyć dodatkowe 2 roboty o różnych nazwach w różnych miejscach

[]:

1.0.3 Zadanie 3

Narysować dowolnym robotem kwadrat wykorzystując serwis /nazwa_robota/teleport_absolute

[]:

1.0.4 Zadanie 4

Usunąć wszystkie roboty.

```
[ ]:
```

```
[ ]:
```

2 Python

```
[ ]: # Zaimportować właściwe biblioteki
```

2.1 Klient

Dla podanych nazw serwisów należy utworzyć klientów, którzy wyślą i odbiorą zapytanie z serwera. Do serwisów testowych wykorzystana została symulacja z turtlesim i 3 turtlebotami z wprowadzenia.

2.1.1 /kill

utworzyć klienta usuwającego żółwia turtle1

```
[ ]:
```

2.1.2 /spawn

utworzyć klienta tworzącego żółwia nowy1

```
[ ]:
```

2.1.3 /turtle1/set_pen

utworzyć klienta dla żółwia o nazwie turtle1 ustawiającego kolor pędzla na czerwony

```
[ ]:
```

2.1.4 /turtle1/teleport_absolute

utworzyć klienta teleportującego żółwia w odpowiednie położenia tak, aby narysował kwadrat

```
[1]:
```

2.1.5 /tb3_1/move_base/make_plan

utworzyć klienta wysyłającego plan do robota. Należy uruchomić symulację z kilkoma robotami z zajęć ze wstępu do ROS (zadanie dodatkowe*)

[2]:

3 Serwer

Dla podanych typów wiadomości serwisowych utworzyć odpowiednie serwery

3.0.1 pkg_tsr/Task

-> współrzędne punktu, rysowanie kwadratu

[]:

```
# utworzyć serwer
```

3.0.2 std_srvs/Empty

-> czyszczenie mapy (to samo co clear, ale napisany własny serwer)

[4]:

```
# utworzyć serwer
```

3.0.3 std_srvs/SetBool

-> utworzyć serwer o nazwie "reset_mapy" resetujący mapę wykorzystujący istniejący serwer reset

[]:

```
# utworzyć serwer
```

3.0.4 std_srvs/Trigger

-> utworzyć serwer, który po wyzwoleniu triggera robot z turtlesim ma narysować trójkąt

[]:

```
# utworzyć serwer
```

4 Zadanie dodatkowe

1. Należy uzupełnić metodę **go_to_goal()** odpowiadającą za dojazd do punktu o podanych współrzędnych. W klasie Robot znajduje się serwis odpowiedzialny za wywołanie tej metody. Serwis korzysta z typu **tsr_materials/Task.srv**. W zapytaniu od klienta w polach x,y

należy przesłać współrzędne celu. W odpowiedzi na zapytanie klienta należy odesłać informację o zakończeniu działania. Metoda powinna wysyłać odpowiednie prędkości sterujące robotem (postępową i obrotową) na topicu `/nazwa_robot/cmd_vel`. Należy ustawić wartość zmiennej od tolerancji dojazdu do celu (np. 0.05) i skorzystać z niej przy dojeżdżaniu do celu. Dodatkowo powinny zostać uwzględnione ograniczenia prędkości (np. 0.3).

2. Analogicznie do metody `go_to_goal` dołożyć inną metodę umożliwiającą dojazd do wskazanego punktu i rozpoczęcie ruchu po dowolnej krzywej np. łamana, spirala, okrąg.

```
[2]: !rosservice call reset
```

```
[ ]: import rospy
from turtlesim.srv import Spawn
from turtlesim.msg import Pose
from geometry_msgs.msg import Twist
from pkg_tsr.srv import Task
import math

rospy.init_node("serwis_node_test")
```

```
[ ]: class Robot:
    def __init__(self, name, x, y):
        self.name = name
        self.follow_service = rospy.Service(name+'/go_to_goal', Task, self.
→go_to_goal)
        self.velocity_pub = rospy.Publisher(name+'/cmd_vel', Twist,
→queue_size=10)
        self.pose_sub = rospy.Subscriber(name+'/pose', Pose, self.callback_pose)

        create_new_turtle = rospy.ServiceProxy('spawn', Spawn)
        create_new_turtle(x, y, 0, name)

    def go_to_goal(self, msg_srv):
        max_speed = 0.3
        tolerance = 0.05
        # UZUPEŁNIĆ - na podstawie informacji o położeniu publikować odpowiednie
→prędkości. Po osiągnięciu
        # celu z zadaną tolerancją zwrócić informację, że zakończono. Do
→określania kierunku ruchu i wyznaczania
        # prędkości mogą przydać się funkcje:
        # - math.atan2(y,x)- zwraca
        # - math.sqrt(x) - zwraca pierwiastek kwadratowy z x

    def callback_pose(self, data):
        # UZUPEŁNIĆ - odebrać i przechować informację o aktualnym położeniu robota
```

```
def unregister(self):  
    self.follow_service.shutdown()
```

```
[ ]: # UZUPEŁNIĆ - utworzyć klienta i zlecić zadanie dojazdu do kilku r  
# różnych punktów  
# UWAGA! po utworzeniu obiektu z robotem o danej nazwie niemożliwe jest  
# utworzenie kolejnych obiektów pod tą samą nazwą. Konieczne będzie  
# usunięcie już istniejących serwisów lub restart symulacji.
```

```
[ ]:
```